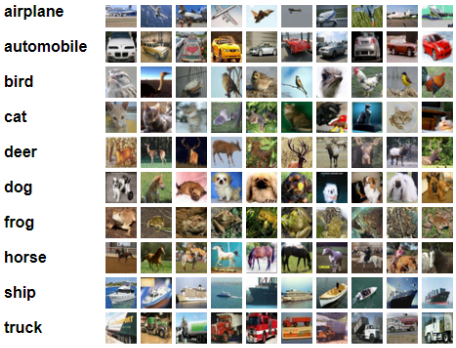
The CIFAR-10 dataset The CIFAR-10 dataset consists of 60000 32x32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images.

The dataset is divided into five training batches and one test batch, each with 10000 images. The test batch contains exactly 1000 randomly-selected images from each class. The training batches contain the remaining images in random order, but some training batches may contain more images from one class than another. Between them, the training batches contain exactly 5000 images from each class.

https://www.cs.toronto.edu/~kriz/cifar.html

Klasifikasi Gambar berwarna dengan Data Set Cifar10




```
!pip install efficientnet
```

```
Collecting efficientnet
  Downloading efficientnet-1.1.1-py3-none-any.whl.metadata (6.4 kB)
Collecting keras-applications<=1.0.8,>=1.0.7 (from efficientnet)
  Downloading Keras_Applications-1.0.8-py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.10/dist-packages (from efficientnet) (0.24.0)
Requirement already satisfied: numpy>=1.9.1 in /usr/local/lib/python3.10/dist-packages (from keras-applications<=1.0.8,>=1.0
Requirement already satisfied: h5py in /usr/local/lib/python3.10/dist-packages (from keras-applications<=1.0.8,>=1.0.7->effi
Requirement already satisfied: scipy>=1.9 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (1.13
Requirement already satisfied: networkx>=2.8 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (3
Requirement already satisfied: pillow>=9.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (10.
Requirement already satisfied: imageio>=2.33 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (2
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientr
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (2
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet)
Downloading efficientnet-1.1.1-py3-none-any.whl (18 kB)
Downloading Keras_Applications-1.0.8-py3-none-any.whl (50 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 50.7/50.7 kB 3.7 MB/s eta 0:00:00
Installing collected packages: keras-applications, efficientnet
Successfully installed efficientnet-1.1.1 keras-applications-1.0.8
```

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from efficientnet.tfkeras import EfficientNetB0
from tensorflow.keras.applications import VGG16
from tensorflow.keras.utils import to_categorical
```

```
# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)

# Preprocess the data
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 13s 0us/step
```

VGG-MODEL

```
# VGG-16 model
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(32, 32, 3))
x = GlobalAveragePooling2D()(vgg_model.output)
x = Dense(1024, activation='relu')(x)
predictions = Dense(10, activation='softmax')(x)
vgg_model = Model(inputs=vgg_model.input, outputs=predictions)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_ke
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
```

```
vgg_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
vgg_model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=10, batch_size=64)
```

```
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 55s 56ms/step - accuracy: 0.2130 - loss: 1.9665 - val_accuracy: 0.3859 - val_loss: 1.4739
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 67s 47ms/step - accuracy: 0.4473 - loss: 1.4105 - val_accuracy: 0.5887 - val_loss: 1.1714
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 46ms/step - accuracy: 0.6103 - loss: 1.0937 - val_accuracy: 0.6825 - val_loss: 0.9143
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 36s 46ms/step - accuracy: 0.7022 - loss: 0.8665 - val_accuracy: 0.7281 - val_loss: 0.7964
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 36s 46ms/step - accuracy: 0.7523 - loss: 0.7288 - val_accuracy: 0.7300 - val_loss: 0.8160
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 41s 46ms/step - accuracy: 0.7919 - loss: 0.6190 - val_accuracy: 0.7421 - val_loss: 0.8098
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 41s 45ms/step - accuracy: 0.8207 - loss: 0.5448 - val_accuracy: 0.7664 - val_loss: 0.7191
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 42s 47ms/step - accuracy: 0.8471 - loss: 0.4643 - val_accuracy: 0.7717 - val_loss: 0.7351
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 46ms/step - accuracy: 0.8587 - loss: 0.4243 - val_accuracy: 0.7609 - val_loss: 0.8188
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 36s 46ms/step - accuracy: 0.8765 - loss: 0.3783 - val_accuracy: 0.7910 - val_loss: 0.7060
<keras.src.callbacks.history.History at 0x7ae62105f460>
```

```
# Evaluate the models
vgg_score = vgg_model.evaluate(x_test, y_test)
print("VGG-16 Model - Test Loss:", vgg_score[0], "Test Accuracy:", vgg_score[1])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - accuracy: 0.7937 - loss: 0.6903
VGG-16 Model - Test Loss: 0.7059550285339355 Test Accuracy: 0.7910000085830688
```

RESNET50 MODEL

```
# InceptionV3 model
from tensorflow.keras.applications import ResNet50
resnet_model = ResNet50(weights='imagenet', include_top=False, input_shape=(32, 32, 3))
x = GlobalAveragePooling2D()(resnet_model.output)
x = Dense(1024, activation='relu')(x)
predictions = Dense(10, activation='softmax')(x)
resnet_model = Model(inputs=resnet_model.input, outputs=predictions)
```

```
resnet_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
resnet_model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=10, batch_size=64)
resnet_score = resnet_model.evaluate(x_test, y_test)
print("Resnet50 Model - Test Loss:", resnet_score[0], "Test Accuracy:", resnet_score[1])
```

```
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 108s 69ms/step - accuracy: 0.5380 - loss: 1.4920 - val_accuracy: 0.2966 - val_loss: 2.7083
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 39ms/step - accuracy: 0.7373 - loss: 0.7907 - val_accuracy: 0.7109 - val_loss: 0.9126
```

```
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 30s 38ms/step - accuracy: 0.7771 - loss: 0.6622 - val_accuracy: 0.5782 - val_loss: 1.3863
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 42s 40ms/step - accuracy: 0.8178 - loss: 0.5500 - val_accuracy: 0.7301 - val_loss: 0.8352
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 39ms/step - accuracy: 0.8440 - loss: 0.4716 - val_accuracy: 0.7110 - val_loss: 0.9658
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 42s 41ms/step - accuracy: 0.8645 - loss: 0.4063 - val_accuracy: 0.7450 - val_loss: 0.9097
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 40ms/step - accuracy: 0.8840 - loss: 0.3513 - val_accuracy: 0.6610 - val_loss: 1.2063
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 41s 40ms/step - accuracy: 0.9004 - loss: 0.3001 - val_accuracy: 0.7235 - val_loss: 0.9544
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 42s 41ms/step - accuracy: 0.8987 - loss: 0.3125 - val_accuracy: 0.7140 - val_loss: 0.9717
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 31s 40ms/step - accuracy: 0.8987 - loss: 0.3192 - val_accuracy: 0.7412 - val_loss: 0.9197
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step - accuracy: 0.7441 - loss: 0.9378
Resnet50 Model - Test Loss: 0.9197342991828918 Test Accuracy: 0.7411999702453613
```

EFFICIENT MODEL

```
# EfficientNetB0 model
efficientnet_model = EfficientNetB0(input_shape=(32, 32, 3), include_top=False, weights='imagenet')
x = GlobalAveragePooling2D()(efficientnet_model.output)
x = Dense(1024, activation='relu')(x)
predictions = Dense(10, activation='softmax')(x)
efficientnet_model = Model(inputs=efficientnet_model.input, outputs=predictions)
```

```
Downloading data from https://github.com/Callidior/keras-applications/releases/download/efficientnet/efficientnet-b0_weights
16804768/16804768 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
efficientnet_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
efficientnet_model.fit(x_train, y_train, validation_data=(x_test, y_test), epochs=10, batch_size=64)
efficientnet_score = efficientnet_model.evaluate(x_test, y_test)
print("EfficientNetB0 Model - Test Loss:", efficientnet_score[0], "Test Accuracy:", efficientnet_score[1])
```

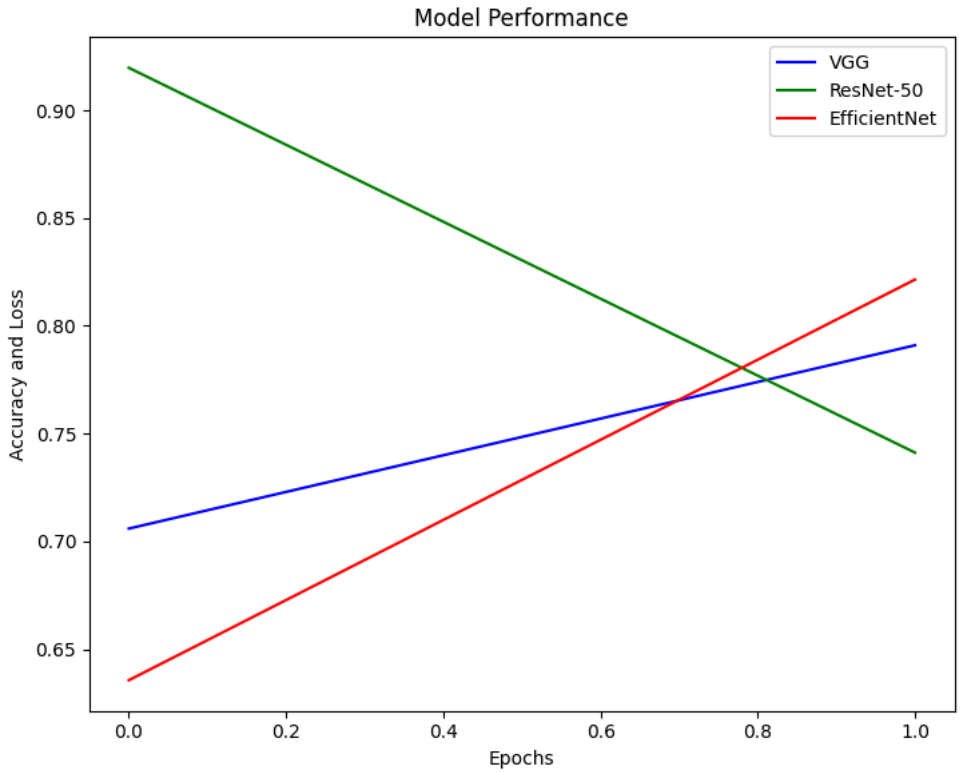
```
Epoch 1/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 127s 78ms/step - accuracy: 0.5602 - loss: 1.3166 - val_accuracy: 0.7624 - val_loss: 0.7105
Epoch 2/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 65s 21ms/step - accuracy: 0.7837 - loss: 0.6535 - val_accuracy: 0.7842 - val_loss: 0.6262
Epoch 3/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 21ms/step - accuracy: 0.8266 - loss: 0.5111 - val_accuracy: 0.8108 - val_loss: 0.6000
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 20s 20ms/step - accuracy: 0.8454 - loss: 0.4623 - val_accuracy: 0.8176 - val_loss: 0.5801
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 18s 23ms/step - accuracy: 0.8742 - loss: 0.3804 - val_accuracy: 0.8220 - val_loss: 0.5784
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 18s 23ms/step - accuracy: 0.8866 - loss: 0.3415 - val_accuracy: 0.8244 - val_loss: 0.5640
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 24ms/step - accuracy: 0.9002 - loss: 0.2947 - val_accuracy: 0.8335 - val_loss: 0.5681
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 18s 21ms/step - accuracy: 0.9112 - loss: 0.2621 - val_accuracy: 0.8181 - val_loss: 0.6285
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 16s 20ms/step - accuracy: 0.9184 - loss: 0.2404 - val_accuracy: 0.8339 - val_loss: 0.5697
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ 21s 21ms/step - accuracy: 0.9302 - loss: 0.2073 - val_accuracy: 0.8215 - val_loss: 0.6356
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.8182 - loss: 0.6533
EfficientNetB0 Model - Test Loss: 0.6356126666069031 Test Accuracy: 0.8215000033378601
```

```
import matplotlib.pyplot as plt

# Data historis untuk akurasi dan loss (Contoh)
epochs = range(1, 10)
# Plot Akurasi
plt.figure(figsize=(14, 6))

# Subplot untuk Akurasi
plt.subplot(1, 2, 1)
plt.plot(vgg_score, label='VGG', color='b')
plt.plot(resnet_score, label='ResNet-50', color='g')
plt.plot(efficientnet_score, label='EfficientNet', color='r')
plt.title('Model Performance')
plt.xlabel('Epochs')
plt.ylabel('Accuracy and Loss')
plt.legend()

# Menampilkan plot
plt.tight_layout()
plt.show()
```


Model Performance
VGG
ResNet-50
EfficientNet
Accuracy and Loss
0.90
0.85
0.80
0.75
0.70
0.65
0.0
0.2
0.4
0.6
0.8
1.0
Epochs